

Context Virtual Memory

Bounded Working Sets for Million-Object Agent Worlds

CVM Project

Working paper ■ 7 October 2026

Abstract

Agents need to investigate large external worlds without placing those worlds in the model prompt. We study *Context Virtual Memory* (CVM), a runtime that gives a model stable object handles, explicit page faults, bounded resident objects, durable notes, and evidence-checked answers. In a synthetic incident world growing from 112 to 999,991 objects, a deterministic prompt-only policy solves 90/90 tasks at each size while its mean peak resident set remains 8.1–9.4 objects and its estimated peak prompt remains 1.55–1.79k tokens. A live DeepSeek-V4.1-Flash processor solves 21/30 tasks at the largest size with 12.3 resident objects and a 1.97k-token estimated mean peak. A controlled reference-policy ablation shows that notes permit perfect accuracy with two resident objects; without notes, two-object accuracy is zero and re-fetching reaches 96%. In a separate live-model test at one million objects, reducing residency from 32 to four objects lowers DeepSeek accuracy from 27/40 to 4/40, accompanied by 59% re-fetching. We also report an interim Qwen2.5-7B-Instruct adapter: 252/256 exact actions on held-out trajectory states after 203 training steps, and a single successful end-to-end serving pilot. These results establish the behavior of this prototype and motivate larger paired model evaluations; they do not establish general world-size invariance or task-level improvement from fine-tuning.

999,991

largest world, objects

1.79kreference peak prompt^a**0.70**live model accuracy, $n = 30$ **4/40**

live accuracy at 4 objects

^aPrompt sizes throughout are characters divided by four, not tokenizer counts.

1 The problem and the claim

An agent can hold a tiny task in a huge world: “find the change that caused this incident” may require a handful of services and facts, even if the environment contains a million objects. Putting every object in a prompt makes cost scale with the world. Ordinary tools avoid that initial dump, but can return whole high-degree objects and leave an unbounded transcript. One-shot retrieval keeps prompts short while denying the agent the chance to choose the next lookup from newly discovered facts.

CVM separates the world address space from the model’s resident view. The runtime owns objects, access rights, replacement, and provenance. The model chooses where to look, which evidence to preserve, and when to answer. Our empirical question is whether *useful behavior* survives a small resident view while the address space grows. A hard cap itself trivially bounds residency; the nontrivial observations are prompt size, accuracy, and re-fetching under that cap.

This paper reports two stages of evidence. V0 measures the substrate and a live model against growing synthetic worlds. V1 measures the four-object failure, constructs training and test data, and reports an unfinished model-training pilot. We present reference-policy and model results separately throughout.

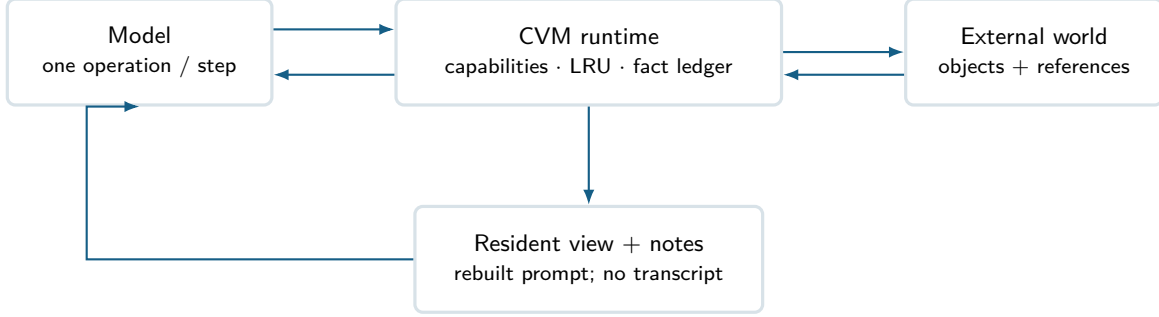


Figure 1: The model issues operations to the runtime, which dereferences external objects and renders a bounded view for the next step. Handles remain usable after eviction; citations refer to fact-ledger entries, not to editable notes.

2 Mechanism

Objects have stable URI-like handles. The model can **SEARCH** for discovery, **TRAVERSE** a named relation, and **FAULT** a known handle into the resident set. A least-recently-used policy evicts objects when the object or token budget is exceeded; the model cannot silently expand the set. **WRITE** stores short, evidence-linked notes in a separately bounded scratch area. **ANSWER** cites fact identifiers previously materialized in that context. The runtime checks namespace capabilities before operations and rejects answers with missing or unmaterialized citations. A rejected answer returns control for another step.

Each model step receives a fresh rendering of task, resident objects, handles, notes, and rules. The previous conversation transcript is not carried forward. In this prototype, the default limit is 32 resident objects and a 16k-token estimated view budget; V1 also evaluates a four-object limit. A task may still use many *total* tokens across many steps. The verifier establishes that cited identifiers were available, not that the model’s inference from them is sound.

3 Experimental design

The primary synthetic world contains clusters of services, hosts, deployments, changes, metrics, incidents, people, teams, and claims. Shared platform services have increasing in-degree as the world grows. Tasks ask for an incident’s root cause, the responsible owner, or whether a claim is supported or contradicted. They require multi-hop dereferences and include temporal and graph distractors. Task depth is one to three in V0. The generator increases *world size* while keeping each sampled task’s local causal chain roughly fixed. The result should therefore be read as a fixed-local-complexity study, not as a theorem for arbitrary graph queries.

The *reference processor* is a deterministic policy that reads only the rendered prompt and emits one action. It has no direct access to the world store. It tests whether the interface carries enough information for a known-good policy; it is not an LLM. V0 compares it with full context, one-shot BM25, graph-expanded retrieval, and a tool-calling baseline that serializes whole objects and preserves the transcript. The latter is a specific by-value API design, not every possible tool design. V0 also runs DeepSeek-V4.1-Flash with a generic prompt, temperature zero, and at most 50 steps. All reported prompt sizes are characters divided by four; model API usage is reported separately in repository artifacts.

4 V0: scaling the world

At five world sizes from 112 to 999,991 objects, the reference processor answers all 90 tasks per tier correctly. Mean peak residency stays between 8.1 and 9.4 objects; estimated peak prompts stay between 1,547 and 1,793 tokens (Figure 2). At the largest size, the whole-object tool baseline also answers all tasks but reaches 116,229 estimated peak prompt tokens. A full-context rendering would require about 79 million estimated tokens at that size; it was not executed there. One-shot BM25 and graph-expanded retrieval score 0.21 and 0.30, respectively, at one million objects, but these are limited, untuned baselines and cannot establish superiority over stronger iterative retrieval.

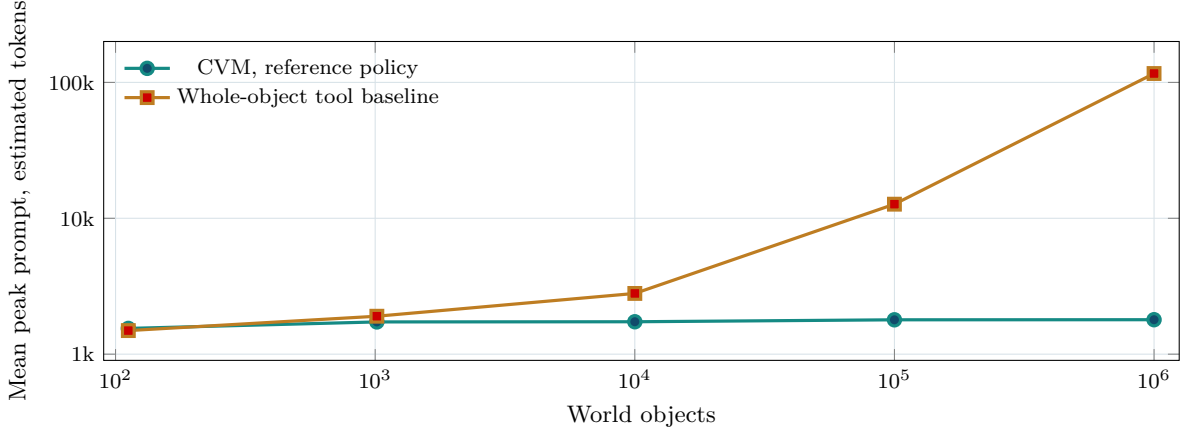


Figure 2: In 90 reference-policy tasks per tier, CVM’s peak prompt stays near 1.5–1.8k estimated tokens. The particular whole-object baseline grows when a lookup serializes the inverse edges of a shared hub. Logarithmic axes; token estimates are characters/4.

Table 1: Live DeepSeek-V4.1-Flash with a 32-object limit. Means are per task. The two accuracy samples are small and use different tasks; the data do not establish equivalence across world sizes.

	1,015 objects	999,991 objects
Tasks	15	30
Correct	8/15 (0.53)	21/30 (0.70)
Mean peak resident objects	11.5	12.3
Mean peak prompt, estimated tokens	1,874	1,970
Mean steps	28.2	29.9
Root-cause accuracy	0.40	0.40

The bounded view has a cost. At one million objects the reference policy takes about 30.5 steps and sends 34,835 estimated prompt tokens in total per task, compared with roughly 9.3 steps for whole-object tool calling. CVM wins on *peak* context in this design; it does not minimize step count or total tokens at every scale. The tool baseline’s growth is caused by serializing a high-degree hub. Returning paginated identifiers would remove much of that particular gap.

The live-model loop retains bounded residency at both sizes (Table 1). Its larger-world accuracy is 21/30, compared with 8/15 in the smaller world. The samples are too small and unpaired to conclude that accuracy improves, or even to tightly bound any degradation. The unchanged 0.40 root-cause accuracy indicates that causal inference and distractor handling are substantial remaining errors. In these live runs, 18–26% of attempted answers were rejected for unsupported citations and retried. No unsupported answer was accepted in the observed traces, but some accepted, cited answers were wrong.

5 Memory pressure: what the notes do

V0 switches `WRITE` notes on and off under the same deterministic policy at roughly one million world objects. With notes enabled, the processor scores 1.00 at resident limits of 2, 4, 8, and 32 objects with no re-fetches. With notes disabled, accuracy falls to 0.00 at two objects, 0.03 at four, and 0.33 at eight; re-fetch rates are 0.96, 0.90, and 0.58, respectively. At 16 or 32 objects, notes-disabled accuracy returns to 1.00. This is a controlled policy ablation: notes let the policy carry discoveries through eviction, but it does not measure what an untrained model would do if forced to write notes.

V1 then tests the pressure on the *same live model and seeded tasks*. DeepSeek drops from 27/40 correct with 32 objects to 4/40 with four; 59% of materializations re-fetch an evicted object, and 22/40 tasks hit the step limit (Table 2). The model writes about one note per task in either condition; the reference processor writes about 12 and succeeds at four objects. Underuse of notes is a plausible explanation for thrashing, supported by the reference

Table 2: V1 live-model test at 999,991 objects: 40 paired seeded tasks, generic prompt, 50-step limit. The four-object reference check uses 30 tasks and a different processor, so its 1.00 is a solvability control rather than a paired model estimate.

	DeepSeek, 32 objects	DeepSeek, 4 objects	Reference, 4 objects
Accuracy	27/40 (0.68)	4/40 (0.10)	30/30 (1.00)
Mean peak resident objects	12.7	3.9	4.0
Re-fetch rate	0.00	0.59	0.00
Notes per task	1.0	0.95	11.6
Step-limit hits	3/40	22/40	0/30

ablation. It is not a causal estimate for DeepSeek: the policies also differ in navigation and answer selection. A direct intervention on the same live model remains necessary.

6 V1: training an operation policy

The V1 exporter runs 10,000 solved reference tasks through the runtime and produces 254,147 training and 50,897 validation prompt/action examples. The world seeds are disjoint across these splits. Examples cover world sizes near 10^2 – 10^4 , three task kinds, depth 1–3, multiple resident limits, and 6,427 recovery targets after invalid answers, denied actions, useless faults, or evictions. This data teaches individual actions from reference trajectories; successful imitation does not by itself imply successful autonomous rollouts.

Seven held-out split generators pass 100/100 reference tasks each, including one-million-object scaling, depth 4–5, traps, a four-object tight split, and a code-repository domain. The second domain uses its own reference policy only as a solvability check; it contributes no training examples and has not yet been evaluated with a live model.

We fine-tune Qwen2.5-7B-Instruct with a rank-32 LoRA adapter. The saved artifact is checkpoint 203 of a planned 1,563 optimizer steps, about 13% of the scheduled epoch. On the same 256 held-out reference-state prompts used for earlier gates, checkpoint 203 emits valid JSON on 255/256, chooses the right operation on 253/256, and matches the exact action on 252/256. In a separate operation-stratified 128-prompt check, it matches 67/69 SEARCH actions and 56/59 ANSWER actions exactly. These samples assess local imitation of a known trajectory distribution. They do not test error recovery after the model’s own mistakes.

A paired serving smoke test uses one held-out four-object task at 999,991 world objects. The base model reaches the 60-step limit without solving it and writes no notes. The step-203 adapter solves it in 30 steps, writes 12 notes, and stays at or below four residents. This is *one task*, a check that serving and evaluation work, not an accuracy estimate or evidence of general improvement. Full base-versus-adapter task-level cells are pending.

7 Relationship to prior work

MemGPT introduced virtual context management for language-model agents and a hierarchy between limited immediate context and larger external memory [1]. MEM1 studies compact model-maintained state across long multi-turn tasks [2]. ClawVM describes harness-managed memory pages, residency, provenance, and thrash metrics for agent sessions [3]. CVM’s mechanisms draw from this line of systems work. Its emphasis here is empirical: an explicit address space of external world objects; measured resident view against *world size*; and a notes ablation under tight object caps. These differences are a description of this prototype’s study, not a claim that paging or agent memory is new.

8 Limits and decisive next tests

The world is synthetic and designed with task-local causal chains. The reference processor is task-specific, so perfect reference accuracy is a capacity check, not evidence of general model ability. The live size comparison has only 15 and 30 tasks and uses different draws. All “prompt token” numbers in the scaling plots are approximate characters/4, and the whole-object and one-shot retrieval baselines leave design and tuning space. The verifier checks citation availability, not semantic entailment; grounded answers can be wrong. The adapter’s 98.4%

exact-action result is conditional on reference states, while its end-to-end comparison has $n = 1$. A code-repository domain is reference-tested only.

The most informative next experiment is a preregistered, paired base-versus-adaptor rollout across world sizes and resident limits, with at least 100 tasks per cell, confidence intervals, and failure analysis. A direct model-side note intervention would test the proposed cause of four-object thrashing. A stronger by-reference tool baseline and iterative retrieval baseline would show whether CVM’s runtime contracts confer benefits beyond API shape. Finally, a real graph or service topology and a faithfulness audit of retried citations would test generality and answer validity.

9 Conclusion

The prototype demonstrates that a model can navigate a million-object address space through a small resident view, and that a deterministic policy can preserve accuracy under a severe object cap when it records useful notes. In the current live model, the four-object setting exposes a sharp failure: repeated materialization and too little retained state. Early imitation training reproduces reference actions and passes a single serving smoke test. The systems result is a measured separation between world size and resident prompt size in these tasks; the open question is how reliably a learned policy can exploit that separation across models and domains.

Artifacts and reproducibility

Code, experiment commands, raw summaries, and traces are in the [2ter repository](#): `cvm/RESULTS_V0.md`, `cvm/RESULTS_V1.md`, `cvm/results/`, and `cvm/experiments/`. V1’s full generated training set is about 1.9 GB and is ignored by Git; `experiments/export_trajectories.py` regenerates it from the recorded seed policy. The Runpod training checkpoint is retained on a network volume; the paper reports only results already recorded in the repository. The local test suite had 41 passing tests at this snapshot.

References

- [1] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>.
- [2] A. Zhou et al. *MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents*. arXiv:2506.15841, 2025. <https://arxiv.org/abs/2506.15841>.
- [3] M. Rafique and L. Bindschaedler. *ClawVM: Harness-Managed Virtual Memory for Stateful Tool-Using LLM Agents*. arXiv:2604.10352, 2026. <https://arxiv.org/abs/2604.10352>.